Experiment No. 2
Perform Edge detection on the MNIST using Markov Random Field (MRFs).
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Perform Edge detection on the MNIST using Markov Random Field (MRFs).

• To understand the concept of Markov Random Fields (MRFs).
• To apply MRF for image processing tasks.
• To perform edge detection on image datasets like MNIST.
• To analyze spatial dependencies between pixels.Objective:

Theory:

Markov Random Field (MRF):

A Markov Random Field is a probabilistic graphical model used to model spatial relationships in data. It is widely used in image processing where each pixel depends on its neighboring pixels.

Unlike Hidden Markov Models, MRFs are undirected graphical models, meaning relationships are mutual and not directional.

Application in Edge Detection: In images:

- Each pixel is considered a node
- Neighboring pixels influence each other
- Edges are detected based on intensity differences between neighboring pixels

MRF helps in:

- Smoothing images
- Detecting edges
- Reducing noise

Energy Function in MRF:

MRF works by minimizing an **energy function**, which consists of:

- **Data term:** Depends on observed pixel values
- **Smoothness term:** Depends on neighboring pixels

Implementation:

- Load MNIST dataset images.
- Preprocess images (grayscale normalization).
- Define neighborhood system (e.g., 4-neighbour or 8-neighbour).
- Construct MRF model representing pixel relationships.
- Define energy function based on pixel intensity differences.
- Apply optimization technique (e.g., Iterated Conditional Modes - ICM).
- Detect edges based on changes in pixel intensities.

Code:

```
# Install dependencies:

# pip install numpy matplotlib tensorflow


import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist


# Step 1: Load MNIST dataset

(x_train, _), (_, _) = mnist.load_data()


# Step 2: Select one image

image = x_train[0].astype(np.float32)


# Step 3: Normalize image

image = image / 255.0
```

```
# Step 4: Define neighborhood (4-neighbour)
def get_neighbors(i, j, shape):
    neighbors = []
    if i > 0: neighbors.append((i-1, j))
    if i < shape[0]-1: neighbors.append((i+1, j))
    if j > 0: neighbors.append((i, j-1))
    if j < shape[1]-1: neighbors.append((i, j+1))
    return neighbors

# Step 5: Initialize labels (0 = no edge, 1 = edge)
labels = np.zeros_like(image)

# Step 6: Define energy function
def energy(i, j, label):
    data_term = abs(image[i, j] - label)

    smoothness_term = 0
    for ni, nj in get_neighbors(i, j, image.shape):
        smoothness_term += abs(label - labels[ni, nj])

    return data_term + 0.5 * smoothness_term
```

```
# Step 7: Apply Iterated Conditional Modes (ICM)
for iteration in range(5):
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            e0 = energy(i, j, 0)
            e1 = energy(i, j, 1)
            labels[i, j] = 0 if e0 < e1 else 1

# Step 8: Display results
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(image, cmap='gray')

plt.subplot(1,2,2)
plt.title("Detected Edges (MRF)")
plt.imshow(labels, cmap='gray')

plt.show()
```

Conclusion:

Markov Random Fields are effective for modeling spatial relationships in images. They help in detecting edges by considering neighboring pixel dependencies, making them useful in image processing tasks.